第 4 组 · 课程实践项目结题答辩

# 智能会议室管理系统

## V1.0　需求分析 · 概要设计 · 详细设计

成员：李知渔　虞业宸　李牧远　孙若普

2026 年 11 月

20 条功能需求　15 条非功能需求

14 条 REST 接口　4 张数据表

43 项回归　50 并发验证

本机运行：Vue 3 + Spring Boot + MySQL

# 目录

CONTENTS


01 需求分析 Requirements Analysis P03 – P07

02 概要设计 High-level Design P08 – P11

03 详细设计 Detailed Design P12 – P16

04 验证与结论 Verification & Conclusion P17 – P19

01

# 需求分析

REQUIREMENTS ANALYSIS

需求清单 V1.1：20 条功能需求、15 条非功能需求

评审意见 6 条全部有落点，范围之外 9 项明确不做

# 需求从哪里来：范围、清单与评审闭环



**需求基线**

需求清单 V1.1 在 V1.0 的 20 条功能需求上逐条标注修改点；15 条非功能需求每条都带验证方式；6 条评审意见全部有处理决定。

**20**

**条功能需求**

FR-01 ~ FR-20

**15**

**条非功能需求**

性能 / 安全 / 可靠性 / 可用性 / 兼容性

**3**

**条质量与交付要求**

QR-01 ~ QR-03，单列不并入功能

## 范围之外，9 项明确不做

人脸识别、物联网设备控制、3D 场景、智能推荐、周期 / 批量预约、多级审批、消息推送、统一身份认证对接、移动端原生应用。

写进文档，也不画进任何图表——避免"综合管理"式的模糊口径。

## 三条编号纪律

① 条目只改状态不重编号；删掉的条目保留行、写明去向

② 评审提了但决定不改的，也留一行（R-06）

③ 同一条规则在角色说明、用例规约、需求清单里写成同一句话

> 需求可验收的标准不是"写全了"，而是每条都能被一个用例和一次测试命中。

# 6 个用例规约，覆盖两条角色主线

## 普通用户 UC-U01 – U05　FR-01、05、07 – 13

### UC-U01　登录与角色隔离

账号密码 + 状态校验；普通用户登录后不出现任何管理入口

### UC-U03　提交预约

必填项、时间规则、会议室状态、时间冲突四道校验，通过即落库无审批

### UC-U04　查看 / 修改我的预约

只显示本人记录；距开始超过 30 分钟才可改，改后重新检测冲突（排除自身）

### UC-U05　取消我的预约

校验归属与 30 分钟时限；取消后时段立即释放，历史记录保留

## 管理员 UC-A03 – A05　FR-03、04、14 – 16

### UC-A03　管理会议室

新增、修改、删除（有预约则拒绝）、启用与停用

### UC-A04　查看和处理预约

按日期、状态、会议室筛选全部预约；强制取消异常预约并填原因

### UC-A05　用户与审计

账号启用 / 停用、角色调整；操作日志可查，普通用户访问返回 403

处理预约 = 强制取消或更正错误记录，

不是逐条审批——方案 A 没有审批环节。

登录 → 查询可用会议室 → **提交预约 • 冲突检测** → 我的预约 → 取消 / 强制取消

# 核心规则：半开区间，首尾相接不算冲突

冲突判定式（FR-09，三份文档同一句话）

**start_new < end_old　且　end_new > start_old**

情形一：首尾相接 —— 允许




情形二：部分重叠 —— 拒绝




## 配套的时间规则

**不跨天**
单次预约只能落在同一天内

**不超过 8 小时**
防止一次占满整天

**开始时间晚于当前**
不允许追溯提交

**改 / 取消提前 30 分钟**
距开始不足 30 分钟即拒绝

这条口径由需求评审 R-04 定下，写进需求清单、用例规约、接口文档与 SQL 判定，四处一致。

# 评审意见逐条落地：5 条采纳，1 条不采纳



| 编号 | 评审发现的问题 | 处理决定与落点 |
| --- | --- | --- |
| R-01 | 停用会议室仍可能被选为新预约 | 采纳 → FR-06 明确停用房间不作为新预约选项 |
| R-02 | 提交预约未写明哪些字段必填 | 采纳 → FR-07 补写主题、人数、联系人 |
| R-03 | 预约时长无上限，可一次约整天 | 采纳 → FR-08 增加不跨天、不超过 8 小时 |
| R-04 | 首尾相接算不算冲突未写明 | 采纳 → FR-09 定为半开区间，相接不算冲突 |
| R-05 | 改 / 取消的时间限制三处表述不一致 | 采纳 → FR-12、FR-13 统一为距开始 30 分钟 |
| **R-06** | 建议增加预约成功短信通知 | 不采纳 → 属消息推送，列入本学期 9 项不做范围 |

### 不采纳也要留痕

R-06 在变更对照表保留一行，写明理由与"本学期不实现"的边界，评审记录可追溯到条目。

### 采纳要落到编号

5 条采纳意见全部改到具体 FR 条目上，后续用例、接口、测试都能反查依据。

02

# 概要设计

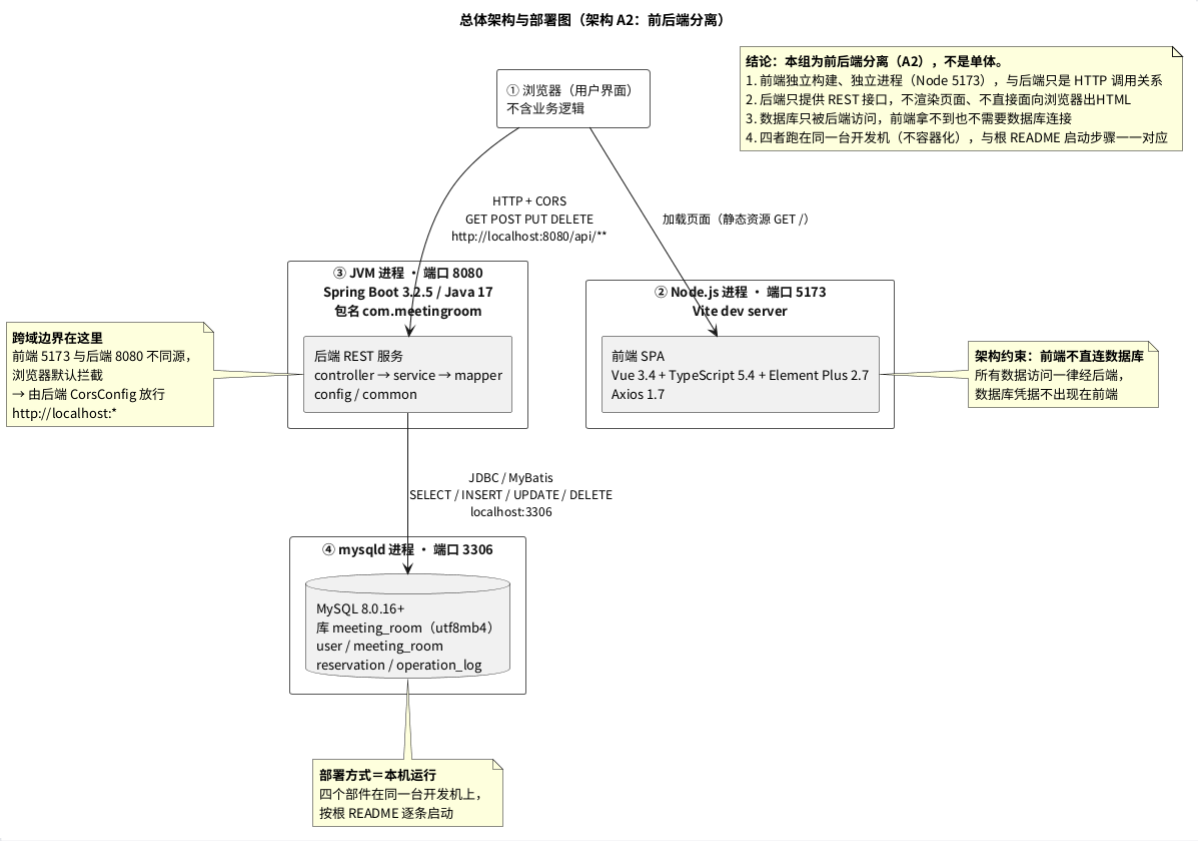
HIGH-LEVEL DESIGN

架构 A2 前后端分离 + 数据库 B1 MySQL

14 条接口冻结，模块与需求的对应关系可逐条反查

# 总体架构：前后端分离（A2）+ MySQL（B1）






图：总体架构与部署（含浏览器、Vite、Spring Boot、MySQL 与端口）

## 选型结论

### A2　前后端分离

Vue 3.4 + TypeScript 5.4 + Element Plus 2.7（Vite 5.2）

Spring Boot 3.2.5 REST + MyBatis（JDK 17）

### B1　MySQL 8.0

InnoDB、utf8mb4；4 张表；CHECK 约束需 8.0.16 及以上

### 单机三进程

Vite 5173　|　JVM 8080　|　mysqld 3306

> 两条硬约束：前端不直连数据库（无任何 JDBC 代码）；跨域由后端 CorsConfig 放行。

> 架构图里的每个部件、端口、启动命令，与根目录 README 的启动步骤一一对应，现场可复现。

# 模块划分：每个模块都挂得上需求编号

## 后端四层（包名 com.meetingroom）

### controller　入口层

Auth / Room / Booking

Admin 系列 4 个控制器

只做接参、转发、包装

FR-01、05、07、11～16

### service　业务层

Auth / Room / Booking

Admin / Log

全部业务规则与事务边界

冲突检测唯一落点

### mapper　数据访问层

User / Room / Reservation

Log 四个 Mapper

只写 SQL，不写规则

冲突查询、占用时段聚合

### config / common　横切

CorsConfig、密码编码器

ApiResponse、异常处理

统一响应与统一异常出口

NF-10、NF-11

## 前端六模块（Vue 3 + TypeScript）

### views

登录、列表、详情

预约、我的、管理端

### router

路由表 + 权限守卫

未登录跳 /login

### store

会话与角色

令牌持久化

### api

统一请求封装

剥壳、判码、中文提示

### types

接口与实体的

TypeScript 定义

### styles

全局样式与

窄屏适配

> 冲突检测只在 BookingService 一处实现——规则唯一落点，便于事务包裹、单测与并发验证。

# 14 条接口冻结：统一响应 + 六条鉴权规则



| # | 接口 | 方法与路径 | 需求 | 鉴权 |
|---|---|---|---|---|
| 1 | 登录 | POST /api/auth/login | FR-01、02 | 公开 |
| 2 | 登出 | POST /api/auth/logout | NF-04 | 登录 |
| 3 | 会议室列表 | GET /api/rooms | FR-05、06 | 登录 |
| 4 | 占用时段 | GET /api/rooms/{id}/occupancy | FR-17 | 登录 |
| 5 | 提交预约 | POST /api/bookings | FR-07 ~ 10 | 登录 |
| 6 | 我的预约 | GET /api/bookings/mine | FR-11 | 登录 |
| 7 | 修改预约 | PUT /api/bookings/{id} | FR-12 | 本人 |
| 8 | 取消预约 | DELETE /api/bookings/{id} | FR-13 | 本人 |
| 9 | 全部预约 | GET /api/admin/bookings | FR-15、19 | 管理员 |
| 10 | 强制取消 | PUT /api/admin/bookings/{id}/cancel | FR-16 | 管理员 |
| 11 | 会议室管理 | GET / POST / PUT / DELETE /api/admin/rooms | FR-03、04 | 管理员 |
| 12 | 用户管理 | GET / POST / PUT /api/admin/users | FR-14 | 管理员 |
| 13 | 操作日志 | GET /api/admin/logs | NF-14 | 管理员 |
| 14 | 利用率概览 | GET /api/admin/stats/utilization | FR-20 | 管理员 |

**统一响应**

{ code, message, data }，code = 200 为成功，前端按 code 判定并翻译中文提示

**鉴权 R1 ~ R6**

缺令牌 401；越权 403；只能操作本人预约；已取消时段立即释放可再约

03

# 详细设计

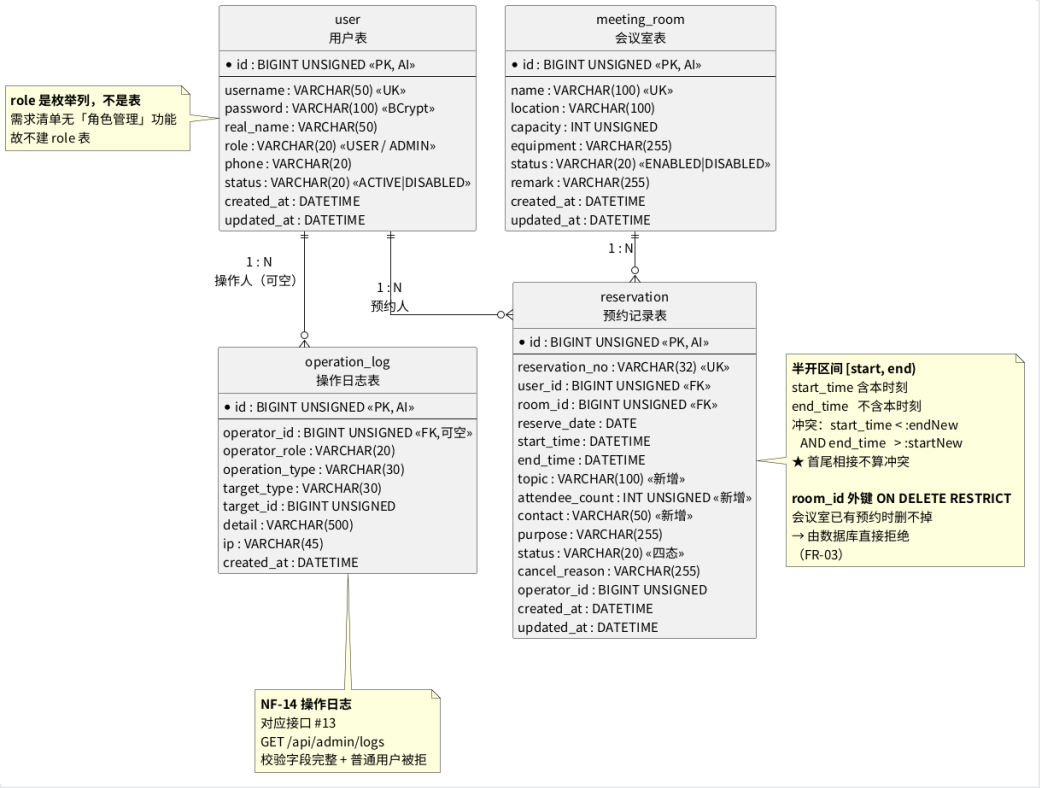
DETAILED DESIGN

4 张表、4 个关键决策、一次事务内的冲突判定

并发场景下只允许生成一条有效预约

# 数据库：4 张表撑起 20 条功能需求






图：E-R 图——user 与 meeting_room 各对 reservation 一对多；user 对 operation_log 一对多

## 四张表的职责

| | |
|---|---|
| **user** | 账号、BCrypt 口令摘要、角色、启停状态 |
| **meeting_room** | 房间名称（唯一）、位置、容量、设备、启用状态 |
| **reservation** | 核心表：房间 × 日期 × 时段、主题/人数/联系人、四态状态 |
| **operation_log** | 操作人、角色快照、对象、结果，供审计（NF-14） |

## 约束与索引

uk_room_name(name)：房间名唯一

idx_reservation_room_time(room_id, reserve_date, start_time)：冲突查询与占用聚合

ck_reservation_date_match：reserve_date 必须等于 start_time 的日期

外键全部 RESTRICT：有预约的房间、有日志的账号都删不掉

> 相对第 6 周骨架只做两处变动：reservation 补 3 个必填业务字段，新增 operation_log——每一处都有需求编号。

# 四个关键决策：为什么这样设计



### ① 外键用 RESTRICT

已产生预约的会议室不允许直接删除，数据库层直接拒绝，接口回 409「存在预约记录，无法删除」。要删，先强制取消相关预约。

### ② reserve_date 冗余存储

按日查询走 idx_reservation_date_status 索引；若写 DATE(start_time) 会让索引失效。冗余字段由同一次写入赋值，另有 CHECK 约束兜底。

### ③ 状态流转不写定时器

需求里没有定时任务功能。待开始 / 进行中 / 已结束在读取时按当前时间推导，更新记录时再校正落库，不引入调度组件。

### ④ 操作日志不级联删除

日志的 operator_id 是普通外键，不设 CASCADE；并冗余存当时角色快照——用户后来升为管理员，历史日志仍如实反映当时身份。

四个决策的共同取向：能用数据库约束保证的，不在应用层"记得做"；能用一次写入保证的，不引入第二个组件。

# 提交预约：一次事务里的冲突判定

1 **参数与时间规则校验**
必填项、不跨天、≤8 小时、开始晚于当前

2 **开事务，锁定会议室行**
SELECT … FOR UPDATE，把同一房间的提交串行化

3 **冲突查询（半开区间）**
排除已取消记录；修改时排除自身；命中则回 409 与占用时段

4 **写入并生成预约编号**
MR + 日期 + 当日序号；写入操作日志

5 **提交事务，返回编号**
任何一步失败整体回滚，不留半条记录

RESERVATIONMAPPER.XML

```
SELECT COUNT(*) FROM reservation
WHERE room_id = #{roomId}
  AND status <> 'CANCELLED'
  AND start_time < #{endNew}
  AND end_time   > #{startNew}
  AND id <> #{excludeId};
```

```
SELECT id FROM meeting_room
WHERE id = #{roomId} FOR UPDATE;
```

### 为什么先锁行再查

只查不锁时，两个事务可能同时查到"无冲突"再各自插入。行锁把同一会议室上的判定串行化，后到的事务看到已提交的记录，自然得到 409。

# 并发防超订：只允许一条预约成功

**1**

**条有效预约**

50 个线程同一秒提交同一房间同一时段

**49**

**条明确失败**

全部返回 409 与占用时段，不是超时或静默失败

**253 µs**

**超时预约校正 SQL**

原全表扫描 3632µs，加日期条件后走 range，约 14 倍

## 怎么测的

测试脚本用 Barrier 让 50 个线程真正同时起跑，目标日期由环境变量指定，跑完核对库内记录数与状态分布，原始响应逐条留档。

## 为什么不会超订

SELECT … FOR UPDATE 把同一会议室行的判定串行化，后到事务能看到已提交记录；失败载荷只回真正重叠的时段，不再把当天全部占用返回。

> 索引改造的等价性由 CHECK 约束 ck_reservation_date_match 保证：reserve_date 必等于 start_time 的日期，加条件不改变结果集。

04

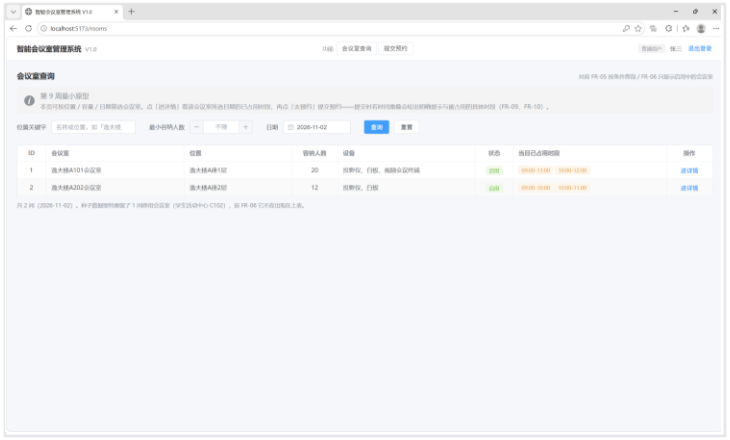
# 验证与结论

VERIFICATION & CONCLUSION

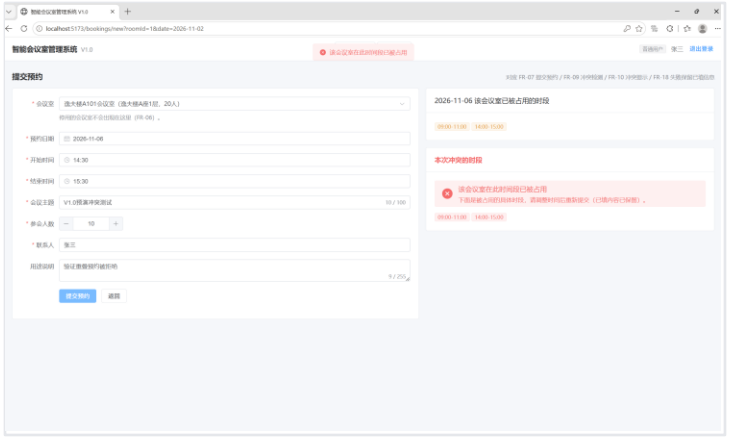
43 项接口回归、功能异常用例、破坏性测试与并发压测

能跑的都在本机跑过，跑不了的如实标出来

# 测试结果与系统演示



登录后菜单按角色渲染，普通用户看不到管理入口



重叠时段提交被拒，页面直接给出已占用时段

**43**

**项接口回归全部通过**

第 10 周 V1.0 现场走通，前后端按 43 项逐条核对

**6 / 6**

**功能异常用例通过**

冲突、跨天、反向时段、首尾相接、停用房间、强制取消

**16 / 16**

**SQL 注入载荷未生效**

4 类载荷 × 4 个入口，全部按普通字符串处理，表结构完好

**1 / 49**

**50 并发下的成功与失败**

一条落库，49 条明确 409，无第二条有效预约

> **如实标出的两项**
>
> 业务失败统一回 HTTP 200、由 body 的 code 区分，与任务书"409"判据不一致（已列入预答辩前修复）；Edge 全流程与干净机部署复现由组内另一位同学补测。

# 结论、局限与下一步

## 已经做到的

① 20 条功能需求全部有接口、有页面、有测试对应，范围外 9 项没有偷偷实现。

② 冲突判定半开区间从需求到 SQL 口径一致，首尾相接可用、部分重叠必拒。

③ 并发提交同一时段只生成一条有效预约，失败一律给出明确原因。

## 还不够的

业务失败的 HTTP 状态码仍是 200，与任务书判据不一致；浏览器兼容与干净机部署两项复现还需补测；性能只在 1000 条量级验证，未做更大数据量压测。

## 下一步

### 状态码口径统一

异常处理器返回对应 HTTP 状态码，前端同步适配冲突提示

### 补齐两项复现

Edge 全流程、清空库后按 README 从零部署

### 数据量级验证

把压测数据扩到万级，复核冲突查询与分页的响应时间

这个系统的价值不在功能多少，而在每条规则都能从需求编号一路查到测试证据。

# 谢谢，请老师和同学批评指正

第 4 组　李知渔　虞业宸　李牧远　孙若普

智能会议室管理系统 V1.0　2026 年 11 月